

Deep Learning

Vision Transformer

Research Depth and Industry Skills

By:

Dr. Zohair Ahmed



www.youtube.com/@ZohairAI

Subscribe



www.begindiscovery.com

Vision Transformer (ViT) – Part of Exam

- An Image Is Worth 16x16 Words:
Transformers For Image Recognition At Scale
- <https://arxiv.org/pdf/2010.11929>

Published as a conference paper at ICLR 2021

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.¹



GELU

- **GELU** is another activation function, like ReLU, but it is **smoother and softer**.
- You already know ReLU:
- $\text{ReLU}(x) = \max(0, x)$
- So:
 - if $x < 0$, output 0
 - if $x > 0$, output x
- That means ReLU makes a **hard decision**:
 - negative? completely block it
 - positive? fully pass it
- **What GELU does**
- **GELU** stands for: **Gaussian Error Linear Unit**
- Its idea is:
 - *instead of making a hard yes/no decision like ReLU, GELU lets values pass in a **smooth, gradual way***
- So GELU is more like:
 - very negative values → strongly suppressed
 - slightly negative values → maybe a little bit kept
 - small positive values → partially kept
 - large positive values → mostly passed through
 - So compared to ReLU, GELU is **less harsh**.

Formula of GELU

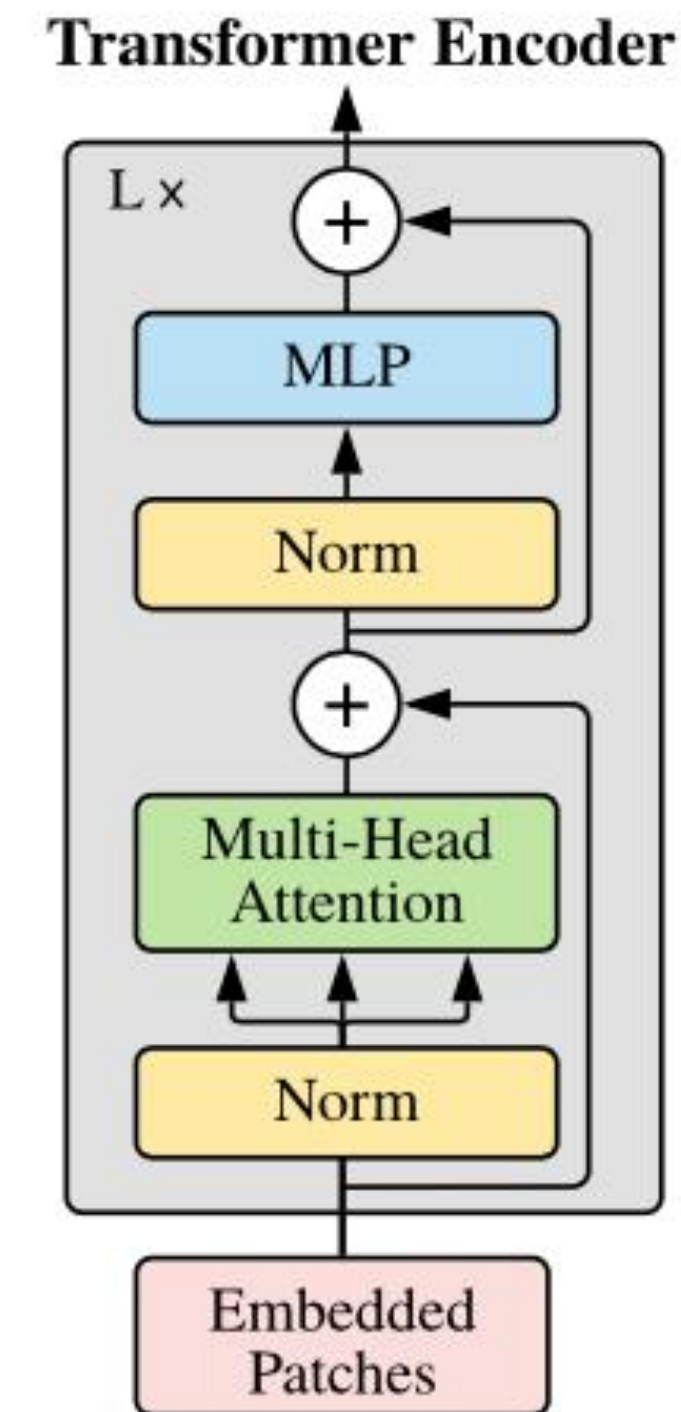
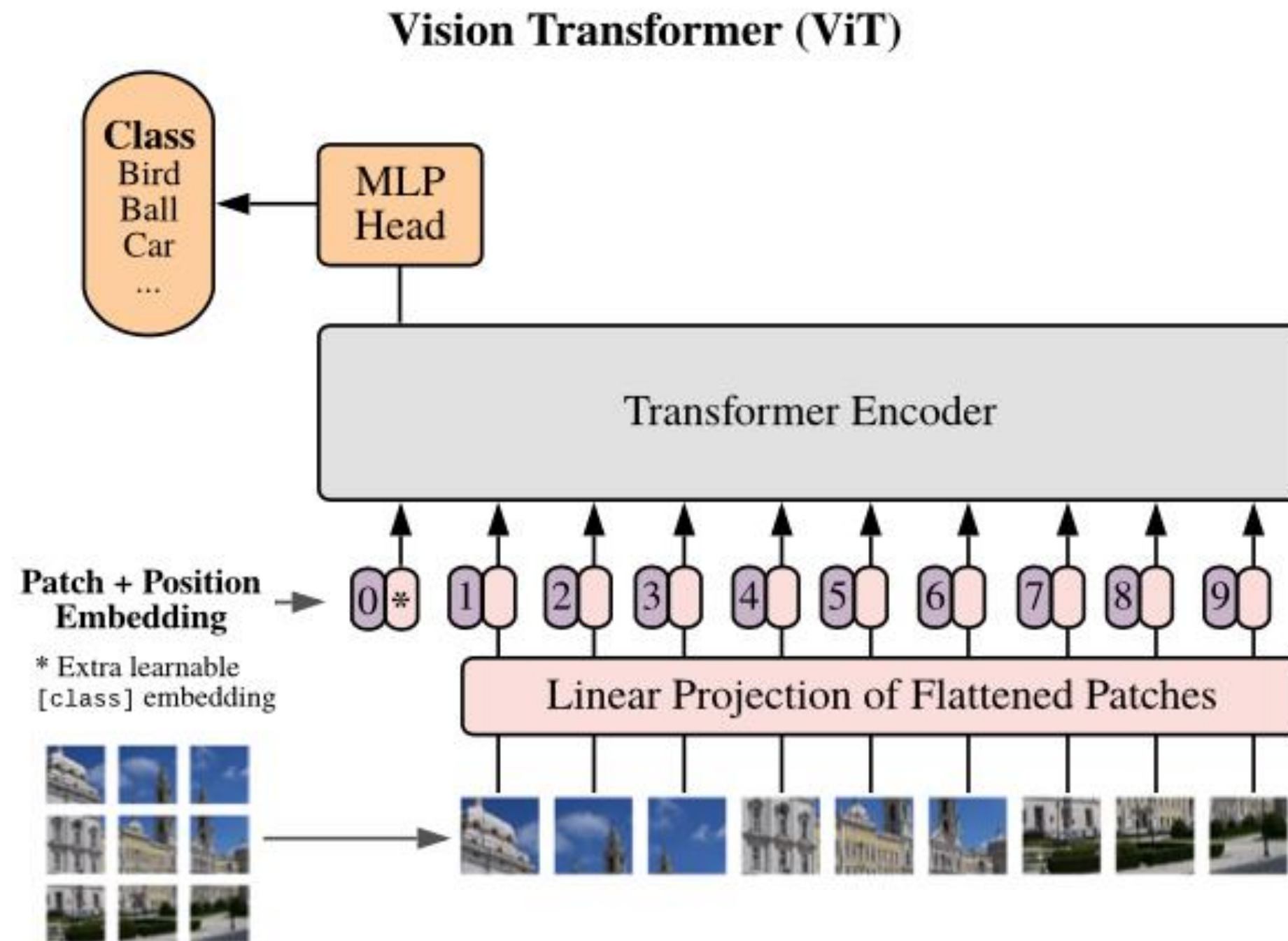
- The exact GELU formula is:
- $GELU(x) = x * \Phi(x)$
- where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution.
- That sounds scary, but meaning simple:
- $\Phi(x)$ gives a number between 0 and 1
- then we multiply x by that number
- So GELU behaves like:
- output = $x \times$ soft gate
- where the gate opens gradually.

Vision Transformer (ViT)

- **Vision Transformer (ViT)** is the idea of using a **transformer to understand images**, just like NLP transformers understand text.
- The biggest idea is this:
- In text, a transformer reads a sentence as a sequence of **tokens**
- In images, ViT reads an image as a sequence of **patches**
- Each patch becomes like a token
- Then the transformer processes those patches to understand the whole image
- So you can think of ViT as:
- Image → small patches → patch embeddings → transformer encoder → image understanding → class/output



Vision Transformer (ViT)



Why this idea is interesting

- Traditional image models like **CNNs** look at local regions with filters.
- ViT takes a different route:
 - break image into patches
 - turn patches into vectors
 - use self-attention to let patches interact with each other
- So ViT can learn relationships like:
 - this patch looks like an eye
 - that patch looks like an ear
 - these two together probably mean “cat”
- It can connect far-away parts of the image more directly than older methods.



Important first fact: ViT usually uses the encoder only

- ViT for image classification usually uses only the transformer encoder
- It does **not** usually use a decoder
- Because most basic ViT tasks are like:
 - “What is in this image?”
 - “Is this a cat, dog, car, or airplane?”
- That is an **understanding** task, not a generation task.
- So ViT mainly needs:
 - read image
 - understand image
 - produce a label
- That is exactly the kind of thing an **encoder** is good at.

Step by Step Flow of VIT



Step 1: Start with an image

- Suppose we have an RGB image:
- $224 \times 224 \times 3$
- This means:
- height = 224
- width = 224
- 3 colour channels = RGB



Step 2: Split image into small patches

- **Suppose patch size is:** 16×16
- Then the image is cut into many small squares.
- **Since:** $224 / 16 = 14$
- **we get:** $14 \times 14 = 196$ patches
- So one image becomes a sequence of:
- 196 visual tokens
- Each patch is like a “word” in the image sentence.

Step 3: Flatten each patch

- **Each patch is:** $16 \times 16 \times 3$
- That contains:
- $16 * 16 * 3 = 768$ values
- So each patch is flattened into a vector of length:
- 768
- So now each patch becomes a 1D vector.



Step 4: Convert each patch to an embedding

- Just like word tokens in NLP become embeddings, patch vectors also become embeddings.
- A linear layer maps each flattened patch into a learned embedding space.
- For example: $768 \rightarrow 768$
- or maybe another dimension depending on the model.
- This is called the **patch embedding**.
- So now each patch is a learned vector representation.



Step 5: Add a special classification token

- ViT usually adds a special token called: [CLS]
- This is similar to the classification token in some language models.
- Its job is:
 - *“collect information from all patches and represent the whole image.”*
- So if there are 196 patches, after adding [CLS], we have: 197 tokens



Step 6: Add positional embeddings

- A transformer by itself does not naturally know:
 - which patch is top-left
 - which patch is center
 - which patch is bottom-right
- Without position, the model would know patch content but not where it belongs.
- So the model now has:
 - patch content embedding
 - position information
- But position matters in images.
- So ViT adds **positional embeddings** to each token.
- This tells the model where each patch came from.

Then the transformer encoder starts

- Now the tokens are ready:
- **[CLS], patch1, patch2, patch3, ..., patch196**
- These go into the transformer encoder layers.
- Each encoder layer usually contains:
- **Multi-head self-attention**
- **Feed-forward network (MLP)**
- **Residual connections**
- **Layer normalization**
- This is very similar to language transformers.

Step 7: Self-attention between patches

- This is the heart of ViT.
- Each patch can “look at” other patches.
- For example:
 - a patch containing part of a wheel can attend to another wheel patch
 - a patch containing fur can attend to face patches
 - a sky patch can attend to bird patches
- So the model learns relationships across the whole image.
- This is powerful because the model is not limited to only nearby pixels.
It can connect distant regions directly.

Step 8: Multi-head attention

- Instead of just one attention mechanism, ViT uses **multiple heads**.
- You can think of each head as focusing on different kinds of relationships:
 - one head may focus on edges
 - another on object parts
 - another on shape consistency
 - another on global structure
- This gives richer image understanding.



Step 9: Feed-forward network

- After attention, each token goes through an MLP:
- Linear $\rightarrow$ GELU $\rightarrow$ Linear
- You already know GELU, so this should feel familiar.
- This helps the model transform token features in a more powerful way.



Step 10: Repeat many encoder blocks

- This process happens again and again across layers.
- So over time, the representation improves:
 - early layers may capture simple textures or boundaries
 - middle layers may capture parts like eyes, wheels, wings
 - deeper layers may capture full object-level meaning



How classification happens

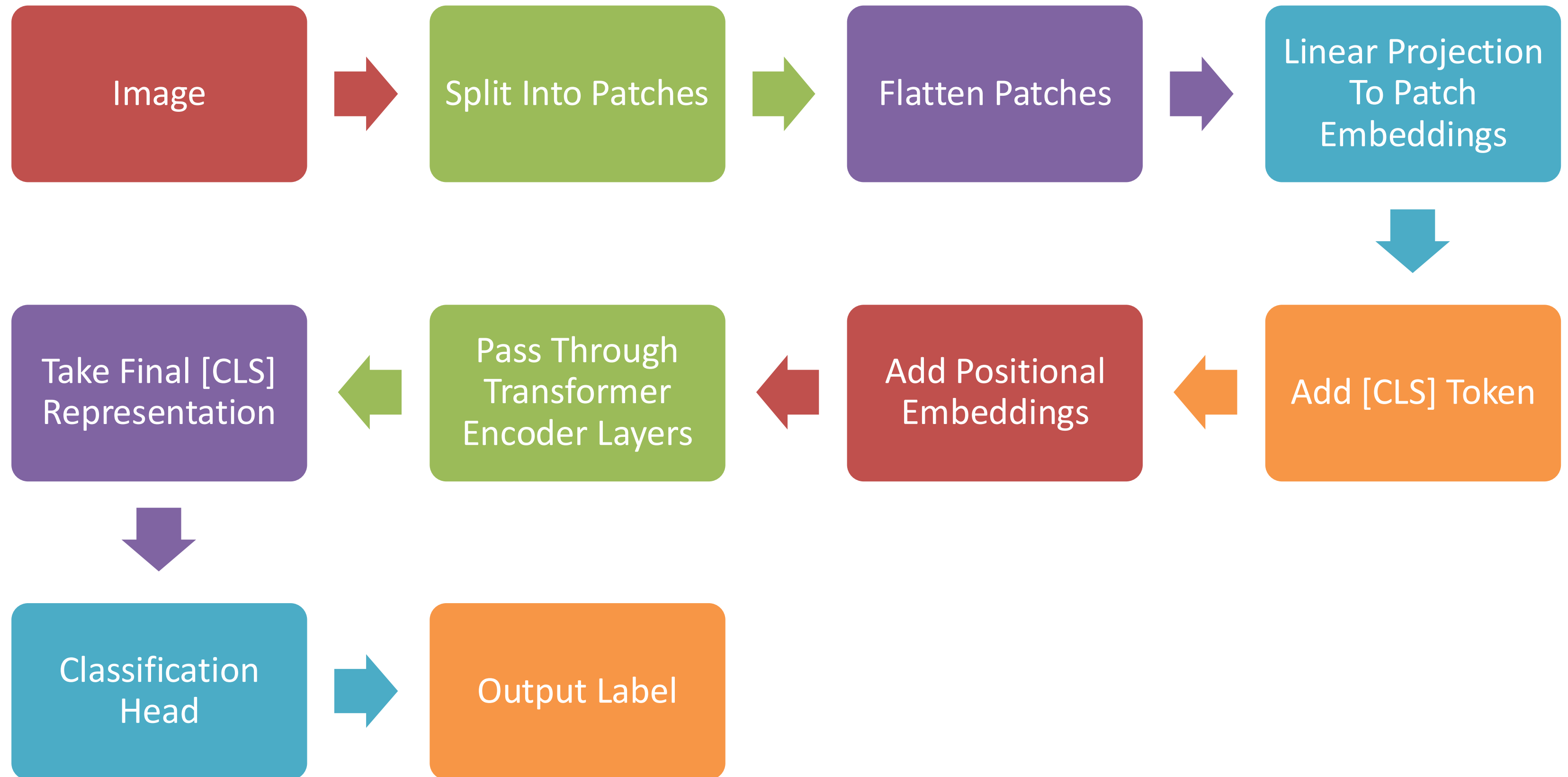
- After the final encoder layer, we take the output of the **[CLS] token**.
- Why [CLS]?
- Because during training, it learns to act like:
 - *“summary of the whole image”*
- Then we send this [CLS] output to a final classifier head, usually a linear layer.
- Example:
- [CLS] output → Linear layer → probabilities for classes

How classification happens

- If classes are:
 - cat
 - dog
 - car
 - bird
- then the final layer may produce:

Class	Probability
cat	0.85
dog	0.08
bird	0.05
car	0.02

Full ViT pipeline in one clean chain



Full ViT

